

RTS2601 — Real-Time Wikipedia SSE Pipeline

A high-pressure real-time data ingestion and analytics pipeline in **Rust**, consuming the live Wikipedia *Recent Changes* SSE firehose

(<https://stream.wikimedia.org/v2/stream/recentchange>).

The system implements the same pipeline twice — once with **Tokio `async/await`**, once with **`std::thread`** — and benchmarks them head-to-head on tail latency, scheduling drift, jitter, and synchronisation throughput.

Built for module **CT087-3-3-RTS** (Real-Time Systems).

Table of Contents

1. [Features](#)
2. [Repository Layout](#)
3. [Prerequisites](#)
4. [Clone and Build](#)
5. [Running the Tests](#)
6. [Running the Pipeline](#)
7. [Running the Benchmarks](#)
8. [Heap Profiling — Zero-Copy Proof](#)
9. [Generating Plots](#)
10. [Demo Script](#)
11. [Mapping to the Assignment Brief](#)
12. [Troubleshooting](#)
13. [License](#)

Features

- **Dual-runtime pipelines:** Tokio (`rts-async`) and `std::thread` (`rts-threaded`) sharing the same core types and metrics.
- **Zero-copy parsing:** `serde` with `Cow<'a, str>` lifetimes; allocations only at the priority-queue boundary. A `dhat-rs` heap profile vs. an owned-string baseline proves the saving.
- **Priority scheduling:** `bot:true` events are low priority; human edits preempt via biased `tokio::select!` (async) or a `BinaryHeap<(Priority, Instant)>` (threaded). Scheduling drift is measured per-event.
- **Drop-oldest backpressure:** bounded ring buffer with a high-precision timestamped *Overflow Event* per drop.
- **Watchdog:** triggers a network reset after 10 s of silence.
- **Fail-Safe / Degraded Mode:** sliding-window p99 jitter trip with hysteresis recovery.
- **Sync-primitive shootout:** Criterion benches comparing `parking_lot::Mutex` , `std::sync::Mutex` , `RwLock` , `DashMap` , and cache-padded atomics across 1–16 threads.
- **Statistical rigour:** `HdrHistogram` p50 / p90 / p99 / p99.9 with sample sizes and 95 % confidence intervals.

Repository Layout

```
Cargo.toml                # workspace root – 6 member crates
rust-toolchain.toml       # pins Rust stable channel
.cargo/config.toml        # release profile: lto=thin, opt-level=3
crates/
  rts-core/               # shared types, Cow parser, DropOldestRing,
                           #   HdrHistogram metrics, FailSafeController
  rts-async/              # Tokio: ingest, biased-select scheduler,
                           #   leaderboard actor, pipeline orchestrator
  rts-threaded/           # std::thread: blocking ingest, BinaryHeap
                           #   priority queue, worker pool
  rts-bench/              # Criterion harnesses
  rts-cli/                # clap entry point – all subcommands
  rts-replay/             # axum SSE server for fixture replay
fixtures/
  recentchange-60s.ndjson  # 3 078 real Wikipedia events (pre-recorded)
scripts/
  demo.ps1 / demo.sh      # end-to-end demo
  plot_latency.py         # tail-latency CDF plot
  plot_shootout.py        # sync-shootout throughput plot
reports/
  runs/   csv/   plots/   dhat/   # results land here
docs/research-report/
```

Prerequisites

You need Rust, Git, and (optionally) Python for plots. Everything else is pulled in by `cargo`.

Rust toolchain

Install via [rustup](#):

```
# Linux / macOS / WSL:
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
```

```
# Windows PowerShell:  
winget install Rustlang.Rustup
```

Verify (any platform):

```
rustc --version    # 1.94 or newer recommended  
cargo --version
```

The repository pins the toolchain via `rust-toolchain.toml` , so the correct channel is selected automatically.

Windows — MSVC Build Tools

Rust on Windows needs the Microsoft C++ linker (`link.exe`).

1. Download from <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
2. In the installer, tick **"Desktop development with C++"** and install.
3. Reboot.

Or via winget:

```
winget install Microsoft.VisualStudio.2022.BuildTools
```

Linux — system packages

```
sudo apt update  
sudo apt install -y build-essential pkg-config libssl-dev git
```

macOS — Xcode CLT

```
xcode-select --install
```

Python (only needed for plots)

Python $\geq$ 3.9, plus three packages:

```
pip install matplotlib numpy hdrh
```

Clone and Build

```
git clone <repository-url> rts2601
cd rts2601
cargo build --workspace --release
```

First build takes 2–5 minutes (downloading and compiling ~80 dependencies). Subsequent builds are seconds.

Always use `--release`. The debug binary is 10–100× slower and benchmark numbers will be wrong.

Running the Tests

```
cargo test --workspace --release
```

Style and lint checks (CI runs these too):

```
cargo fmt --check
cargo clippy --workspace --all-targets -- -D warnings
```

Running the Pipeline

The CLI binary is `rts` (built into `./target/release/rts`). All examples below assume you are in the project root.

There are two ways to drive the pipeline: against the **live** Wikimedia stream, or against a **local replay** of a pre-recorded fixture (deterministic, no network needed).

Option A — Live Wikimedia stream

```
# Async pipeline:
cargo run --release -p rts-cli -- run-async --duration 60s

# Threaded pipeline:
cargo run --release -p rts-cli -- run-threaded --duration 60s
```

Ctrl+C stops it early.

Option B — Local replay (recommended)

The fixture `fixtures/recentchange-60s.ndjson` contains 3 078 real Wikipedia events. Replay makes runs reproducible and works without internet.

Terminal 1 — replay server:

```
cargo run --release -p rts-cli -- replay play \
  --fixture fixtures/recentchange-60s.ndjson \
  --rate 10x \
  --port 8080
```

The server prints `Replay server listening on 127.0.0.1:8080`. Leave it running.

Terminal 2 — run a pipeline against the replay:

```
# Async:
cargo run --release -p rts-cli -- run-async \
  --url http://127.0.0.1:8080/v2/stream/recentchange \
  --duration 60s \
  --log-path reports/runs/async_replay.ndjson \
  --metrics-path reports/csv/async_replay
```

```
# Threaded:
cargo run --release -p rts-cli -- run-threaded \
  --url http://127.0.0.1:8080/v2/stream/recentchange \
  --duration 60s \
  --log-path reports/runs/threaded_replay.ndjson \
  --metrics-path reports/csv/threaded_replay
```

`--rate` accepts `1x`, `10x`, `100x`, or `max` (no inter-event delay).

Recording a fresh fixture

```
cargo run --release -p rts-cli -- replay record \  
  --duration 60s \  
  --out fixtures/my_recording.ndjson
```

CLI reference

```
rts run-async      [--url URL] [--duration 60s] [--log-path PATH] [--metrics-path PATH]  
rts run-threaded   [--url URL] [--duration 60s] [--log-path PATH] [--metrics-path PATH]  
rts replay record  --duration 60s --out PATH  
rts replay play    --fixture PATH [--rate 1x|10x|100x|max] [--port 8080]  
rts stress         (multiplier sweep — see scripts/demo.sh)
```

Running the Benchmarks

Benchmarks take 10–15 minutes each. For canonical numbers run them on Linux or WSL — the Windows scheduler adds 50–200 μ s of context-switch noise.

Sync-primitive shootout

```
cargo bench -p rts-bench --bench sync_shootout
```

Outputs:

- `target/criterion/` — HTML report (open `target/criterion/<group>/report/index.html`)
- `reports/csv/sync_shootout.csv` — raw numbers for plotting

Tail-latency comparison (async vs threaded)

```
cargo bench -p rts-bench --bench tail_latency
```

Outputs:

- `target/criterion/` — HTML report
- `reports/csv/tail_latency.csv` — p50 / p90 / p99 / p99.9 per runtime $\times$ batch size

Quick sanity check (~30 s instead of 15 min)

```
cargo bench -p rts-bench --bench sync_shootout -- --warm-up-time 1 --sample-size 10
```

Heap Profiling — Zero-Copy Proof

The `dhat-heap` feature compiles in a heap profiler. Comparing two runs proves the `Cow<'a, str>` saving:

1. **Zero-copy build** (default `Cow` parsing).
2. **Owned-string baseline** (the `owned-baseline` feature forces `.to_string()` on every field).

Run the replay server in another terminal first, then:

```
# Run 1 — zero-copy:
cargo run --release --features dhat-heap -p rts-cli -- run-async \
  --url http://127.0.0.1:8080/v2/stream/recentchange \
  --duration 60s
mv dhat-heap.json reports/dhat/zero_copy.json

# Run 2 — owned baseline:
cargo run --release --features "dhat-heap,owned-baseline" -p rts-cli -- run-async \
  --url http://127.0.0.1:8080/v2/stream/recentchange \
  --duration 60s
mv dhat-heap.json reports/dhat/owned_baseline.json
```

Open both JSON files in the [dhat viewer](#) (drag and drop) and compare `total_bytes`. Zero-copy is roughly 60× smaller on the parser hot path.

The pipeline also exposes a runtime borrowed/owned counter via `rts_core::event::cow_stats()` — Wikipedia's mostly-ASCII payload yields > 99 % borrowed parses.

Generating Plots

After running the pipeline / benchmarks so that CSVs exist under `reports/csv/` :

```
python scripts/plot_latency.py
python scripts/plot_shootout.py
```

PNGs land in `reports/plots/`.

Demo Script

The demo runs everything end-to-end in ~5 minutes using only the local replay (no internet). It shows:

1. Async pipeline under 10× burst load.
2. Threaded pipeline for comparison.
3. CPU stress injection → Degraded Mode trips → recovery.
4. Replay server killed → watchdog reset fires.
5. Final stats snapshot.

```
# Linux / macOS / WSL:
chmod +x scripts/demo.sh
./scripts/demo.sh
```

```
# Windows PowerShell:
.\scripts\demo.ps1
```

Add `--skip-build` (`-SkipBuild` on PowerShell) if you have already built the workspace.

Mapping to the Assignment Brief

Brief requirement	Where it lives
Dual pipeline (async + threaded)	<code>crates/rts-async/</code> , <code>crates/rts-threaded/</code>
Bounded channel + drop-oldest + overflow event	<code>rts_core::channel::DropOldestRing</code>
Zero-copy <code>serde</code> parsing with <code>Cow<'a, str></code>	<code>rts_core::event::Event<'a></code> , <code>parse_one</code>

Brief requirement	Where it lives
Bot-vs-human priority scheduling	<code>rts_core::priority::Priority</code> , <code>biased select!</code> / <code>heap</code>
2 ms micro-deadline + scheduling-drift metric	<code>rts_core::metrics</code> , per-event drift histogram
Top-3 domain leaderboard (shared resource)	leaderboard actor / shared map in each pipeline crate
Mutex / RwLock / Atomic synchronisation benchmark	<code>crates/rts-bench/benches/sync_shootout.rs</code>
10 s watchdog → network reset	<code>rts_core::watchdog</code>
Fail-Safe / Degraded Mode with hysteresis	<code>rts_core::failsafe::FailSafeController</code>
Heap-allocation proof (zero-copy)	<code>dhat-heap</code> feature; <code>reports/dhat/</code>
Tail-latency p50/p90/p99/p99.9 (async vs threaded)	<code>crates/rts-bench/benches/tail_latency.rs</code>
Research report (3000–4000 words)	<code>docs/research-report/report.md</code>
Demonstration	<code>scripts/demo.ps1</code> , <code>scripts/demo.sh</code>

Troubleshooting

cargo: command not found

Rust isn't on `PATH` .

- Linux/macOS/WSL: `source "$HOME/.cargo/env"` .
- Windows: close and reopen the terminal, or add `%USERPROFILE%\cargo\bin` to `PATH`.

Compile errors after pulling

```
cargo update
cargo build --workspace --release
```

Port 8080 already in use

```
# Linux / macOS / WSL:  
lsof -i :8080  
kill <pid>
```

```
# Windows:  
netstat -ano | findstr :8080  
taskkill /PID <pid> /F
```

Or pass `--port 9090` to both the replay server and the pipeline's `--url` .

Pipeline exits with `connection refused`

The replay server isn't running. Start it in a separate terminal first.

Benchmark numbers look unreasonably slow

- Confirm `--release` is on the command line.
- On Windows, run the benchmarks under WSL2 — the NT scheduler adds noticeable noise.
- Add the project folder to your antivirus exclusion list (`target/` churns a lot).

`dhat-heap.json` not produced

You need both `--features dhat-heap` **and** a clean exit (let the duration finish; don't kill with `Ctrl+C`).
The profile is flushed on drop.

Python plots fail with `ModuleNotFoundError`

```
pip install matplotlib numpy hdrh
```

License

MIT OR Apache-2.0.